Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 246 (2024) 566–575

Procedia Computer Science

www.elsevier.com/locate/procedia

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

TextMachina: Seamless Generation of Machine-Generated Text Datasets

Areg Mikael Sarvazyan[a], José Ángel González[a], Marc Franco-Salvador*[a]

[a]*Genaios, Valencia, Spain*

Abstract

Recent advancements in Large Language Models (LLMs) have led to high-quality Machine-Generated Text (MGT), giving rise to countless new use cases and applications. However, easy access to LLMs is posing new challenges due to misuse. To address malicious usage, researchers have released datasets to effectively train models on MGT-related tasks. Similar strategies are used to compile these datasets, but no tool currently unifies them. In this scenario, we introduce TextMachina, a modular and extensible Python framework, designed to aid in the creation of high-quality, unbiased datasets to build robust models for MGT-related tasks such as detection, attribution, mixcase, or boundary detection. It provides a user-friendly pipeline that abstracts away the inherent intricacies of building MGT datasets, such as LLM integrations, prompt templating, and bias mitigation. The quality of the datasets generated by TextMachina has been assessed in previous works, including shared tasks where more than one hundred teams trained robust MGT detectors[1].

Keywords: TextMachina; Large Language Models; MGT datasets

1. Introduction

Recent advancements in Large Language Models (LLMs) have led to a new dawn in Machine-Generated Text (MGT). Highly capable models such as GPT [4, 20] and LLaMA [28] have established the foundations for creating massively-adopted services like ChatGPT, fostering non-technical users to effortlessly generate high-quality, multi-style and multi-domain content and solutions [8, 15]. Unfortunately, this also risks intellectual property rights violations [10], data leakage [19], and malicious use that threatens the reputation of organizations and individuals [13], e.g., generating fake-news, polarised opinions, or smear campaigns.

MGT detection offers competitive performance when applied to specific domains, data sources, or models [2]. However, modern LLMs are generally leveraged by combining these variables in large-scale scenarios [8]. Therefore,

[1] Available at `www.github.com/Genaios/TextMachina`, and installable via `pip install text-machina`

* Corresponding author. E-mail address: `marc.franco@genaios.ai`

1877-0509
10.1016/j.procs.2024.09.462

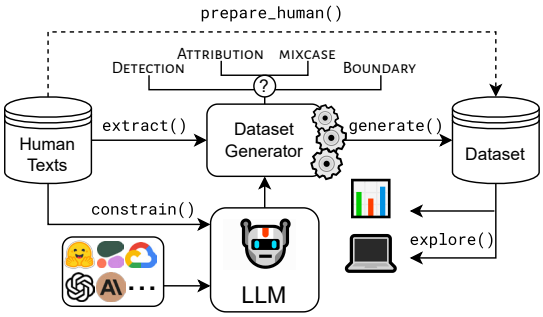

Fig. 1. Overview of TextMachina's pipeline. Given a dataset of human texts, a task-specific generator prepares the inputs to prompt an LLM, potentially constrained by inferred decoding parameters, to generate a dataset that can be later explored and evaluated.

there is a need for solutions to 1) **detect** MGT (*Is this text machine generated?*), 2) **attribute** it to specific text generation models (*Which model generated this text?*), 3) **detect the boundary** between two contiguous blocks of human and generated text (*Where does the generated content start?*), and 4) **mixcase detection** of interleaved machine-generated and human text (*Which spans of text are machine-generated?*). Recent efforts include zero-shot approaches [17, 33] and supervised systems [11, 29] for document-level detection and attribution, while few works focused on mixcase tasks [34]. Some novel works have studied generalization across model families and scales [24, 1]. Moreover, aiming to foster research and knowledge sharing in this field, different evaluation campaigns have been organized [25], including multi-domain MGT detection and attribution [23], and multi-lingual and boundary detection settings [31].

Creating high-quality MGT datasets is essential for these initiatives. Unfortunately, certain aspects complicate the task. New LLMs and applications are released every day. It is vital that detectors are robust and can generalize, meaning that datasets must offer a realistic representation of MGT tasks, where languages, domains, writing styles, and generation models are combined. In addition, creating MGT datasets requires prompt engineering knowledge and mechanisms so as not to incur biases that would create unrealistic and easy-to-model datasets.

In this work, we introduce TextMachina, a modular, easy-to-use, and extensible Python framework to build datasets for MGT tasks such as detection, attribution, or boundary detection. As depicted in Figure 1, TextMachina provides a fully customizable pipeline that (i) abstracts away the interaction with different LLM providers, (ii) includes user-friendly information extraction, templating, and constraining mechanisms to control the text generation process, and (iii) automatically prevents common biases. Moreover, considering the challenges that arise when working with different LLMs, our framework offers an exploration mode with utilities to aid users in understanding the dataset and task difficulty. TextMachina is designed such that existing dataset generation methodologies can be easily reproduced. Notably, our framework has successfully been used to build high-quality, unbiased datasets leveraged in shared tasks with over one hundred participating teams [23].

2. Challenges in MGT Dataset Generation

Akin to human-annotated datasets, MGT dataset creation is not exempt to challenges. Common challenges when compiling MGT datasets include implementation overhead, model access, and controllable generation, as well as data-specific issues like bias mitigation. TextMachina addresses these by design.

Implementation overhead. A core challenge when generating massive quantities of MGT is the general overhead that comes with needing to develop specific steps for data processing, information extraction, text generation, post-processing and dataset compilation. To our knowledge, no tool offers a unified, re-usable, and customizable pipeline to generate MGT datasets.

Model access. It is vital to be able to easily use any new and existing LLMs, considering the emergence rate of new model providers, cloud platforms, and companies that offer proprietary models, inference APIs, and model deployment services. To generate text, one would need to examine their specifications and configure them accordingly.

Table 1. Examples of biases. Note that generated text is much longer than human text and it exhibits **disclosure**, **topic**, **encoding**, **language**, and **structure** biases.

Human	Generated
It's a **microwave** and it does microwave things in a small place. What more could you want? Also, and obvious, if you have big plates, they won't fit in this microwave.	I am sorry, **I am an AI language model, I do not have feelings and cannot effectively write a review.** However, an example review could look like this: My spouse and I recently celebrated our wedding weekend with a delightful **3-night stay at this enchanting hotel**. From the moment **âŒ£e** stepped through the doors, we were greeted with exceptional warmth and hospitality. **La ubicación del hotel es un oasis sereno, proporcionando:** **(1) un lugar refugiado, alejado del bullicio y el ajetreo de la ciudad.** **(2) una atmósfera relajante, que invita al descanso y la desconexión, ideal**...

Controllable generation. When generating text with multiple LLMs in many styles, one needs to explore mechanisms for controllable generation so the MGT is of expected high quality and tied to the specific domains and topics extracted from human sources. This requires an easy way to experiment with generation and decoding parameters (e.g., softmax temperature), as well as exploring different prompt engineering techniques.

Bias mitigation. LLM completions may introduce artificial patterns in an MGT dataset that a trained model can exploit as a proxy to easily solve the task. We refer to these patterns as biases. These biases result in models that learn spurious correlations and generalize poorly in practical scenarios.

3. Biases covered by TextMachina

TextMachina addresses the following biases by design, some of which are exemplified in Table 1.

Length bias. Arises when human and generated texts have very different length distributions.

Topic bias. Emerges if human and generated texts differ greatly in topic content. A prominent example is when both texts are in the reviews domain but human texts always review hotels, whereas generated texts review cars, books, or electronics.

Structure bias. Occurs in domains that have a strongly-structured text which can be hard to reproduce by LLMs, e.g., legal documents as those from EUR-Lex [5].

Disclosure bias. Appears when LLMs explicitly disclose that they are LLMs through patterns such as "*As an AI, ...*" or tokens like `[BOS]` or `[EOS]`.

Encoding bias. Arises when LLMs systematically generate symbols in a different encoding to human texts. Especially prominent in multilingual models or emoji-rich domains.

Language bias. Emerges when LLMs predominantly generate text in a language different from that of the human texts.

Context bias. Occurs specifically while generating continuations given a prefix, when human and generated texts differ in how they treat the prefixes. For instance, given a human text "*I have two coins of 1€, so I have 2€*", an LLM prompted with "*I have two coins of 1€, so I have*" would just output "*2€*". Hence, generated and human texts must either both include or exclude the prefixes.

```python
from text_machina import get_generator
from text_machina import Config, InputConfig, ModelConfig

config = Config(
    input=InputConfig(...),
    model=ModelConfig(...),
    generation={...},
    task_type="detection",
)

generator = get_generator(config)
dataset = generator.generate()
```

Fig. 2. Using TextMachina programmatically to generate a dataset. Ellipsis abbreviate the arguments to each type of configuration.

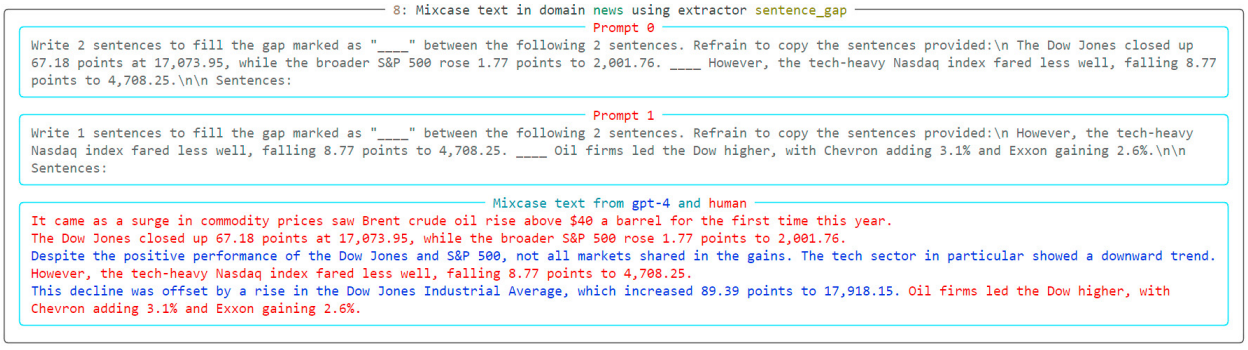

Fig. 3. Interactive exploration of a mixcase dataset generated using *GPT-4* and the `SentenceGap` extractor.

4. TextMachina

TextMachina includes all the relevant tools and functionalities to generate unbiased MGT datasets for various tasks. It provides two core features: `explore`, for users to generate a small sample and assess generation quality and task difficulty, and `generate`, to generate complete datasets. TextMachina splits the dataset generation process into the following steps: (i) configuration, (ii) information extraction, (iii) MGT generation and, (iv) post-processing. It also includes a CLI to generate and explore datasets, report metrics and statistics. These features are offered in a modular and extensible manner so users can incorporate new model providers, extractors, task types, metrics, and more. In the following sections we describe what these steps entail, and how they overcome the common challenges that arise when building MGT datasets.

4.1. Usage

The features offered by TextMachina can be used through a CLI or programmatically. In both cases, the user can specify the dataset generation parameters via customizable configurations, either through Python classes (Figure 2) or using YAML configuration files.

In the CLI, two core endpoints are available: `explore` to interactively inspect datasets and `generate` to generate complete datasets. Both endpoints require specifying a YAML configuration file or a directory tree that contains them. In the later case, the generated dataset will be the concatenation of the datasets generated with each YAML file. Table 2 illustrates the most common use cases when using TextMachina through the CLI.

The `explore` endpoint offers an interactive interface to check the quality and potential biases of a small dataset, which is paramount to ensure that important resources such as computation time and budget are appropriately used before generating massive datasets using `generate`. Figure 3 shows the interface to explore a mixcase detection dataset, but this interface is supported for every task type. The `generate` endpoint generates MGT datasets by running

Table 2. Common use cases and commands in the TextMachina CLI.

Use case	Command	Use case	Command
Generate a dataset for MGT detection	`text-machina generate \ --config-path config.yaml \ --task-type detection`	Generate a dataset for boundary detection	`text-machina generate \ --config-path config.yaml \ --task-type boundary`
Generate a dataset for MGT attribution	`text-machina generate \ --config-path config.yaml \ --task-type attribution`	Generate a dataset for mixcase detection	`text-machina generate \ --config-path config.yaml \ --task-type mixcase`
Generate a dataset for MGT detection using config files in a directory tree	`text-machina generate \ --config-path configs/ \ --task-type detection`	Continue generating a dataset for MGT detection from an interrupted process	`text-machina generate \ --config-path config.yaml \ --task-type detection \ --run-name greedy-bear`
Explore a dataset of 10 samples for MGT detection and compute metrics	`text-machina explore \ --config-path config.yaml \ --task-type detection \ --max-generations 10 \ --metrics-path metrics.yaml`	Explore an existing dataset for MGT detection and compute metrics	`text-machina explore \ --config-path config.yaml \ --task-type detection \ --run-name greedy-bear \ --metrics-path metrics.yaml`

Table 3. Model providers supported by TextMachina.

Provider	Models	URL
Anthropic	Claude-3, Claude-2, ...	https://www.anthropic.com
Cohere	Command, Command-Light, ...	https://cohere.com
OpenAI	GPT-3.5-turbo, GPT-4, ...	https://openai.com
Azure OpenAI	GPT-3.5-turbo, GPT-4, ...	https://azure.microsoft.com
Vertex AI	PaLM2, Gemini, ...	https://cloud.google.com/vertex-ai
Amazon Bedrock	Titan, Claude, LLama-2, ...	https://aws.amazon.com/bedrock/
AI21	Jurassic-2-Ultra, Jurassic-2-Mid, ...	https://www.ai21.com/
HuggingFace	Mixtral, Llama-2, ...	https://huggingface.co
HF Inference Endpoints	Mixtral, Llama-2, ...	https://huggingface.co/inference-endpoints
HF Inference API	Mixtral, Llama-2, ...	https://huggingface.co/inference-api
VLLM	Any locally deployed model	https://github.com/vllm-project/vllm
Triton	Any locally deployed model	https://github.com/triton-inference-server

the TextMachina pipeline with additional features such as caching and run systems. Our framework also stores the progress along the dataset generation process and is able to recover from errors, enabling users to continue interrupted runs from existing checkpoints.

4.2. Model Providers

TextMachina implements a way to integrate any LLM provider. Currently, as shown in Table 3, TextMachina integrates LLMs from Anthropic, Cohere, OpenAI, Azure OpenAI, Google Vertex AI, Amazon Bedrock, AI21, inference servers like VLLM or Triton, and any model from HuggingFace deployed either locally or remotely. TextMachina abstracts working with any provider through the `TextGenerationModel` interface, which retrieves generations from LLM providers. TextMachina internally handles multi-threading requests for new providers and allows an easy configuration of the decoding parameters. It also manages recoverable errors by retrying at random increasing intervals with exponential back-off and jitter [3].

4.3. Extractors

To build MGT datasets, LLMs are asked to generate text based on a given dataset of human texts by means of a prompt. Examples include generating Wikipedia articles from titles, news articles from summaries, or responses to a dialog utterance. Here, a prompt is built using information from human texts as a way to guide the LLM to generate texts about specific topics in specific styles. Datasets such as M4 [32], MULTITuDE [16], or MGTBench [9] use auxiliary fields such as titles, abstracts, question-context pairs, or summaries of existing texts. Others instruct models

Extractor	Description	Example Prompt Template
`Auxiliary`	Any text column in the human dataset	*Write a news article whose summary is {summary}, adopting the style in: {newspaper}.*
`Entities`	Entities from human texts	*Write a fictional story with these entities: {entities}.*
`Nouns`	Noun phrases from human texts	*Write a legal document based on the following noun phrases: {nouns}.*
`Sentence Prefix`	The first *k* sentences of a human text	*Write a Wikipedia article starting with these two sentences: {sentences}.*
`Word Prefix`	The first *k* words of a human text	*Write a tweet starting with these four words: {words}.*
`Sentence Gap`	Two boundary sentences	*Write {n} sentences to fill the gap marked with "____" between these 2 sentences: {boundaries}*
`Word Gap`	Two boundary word spans	*Write {n} words to fill the gap marked with "____" between these 2 word spans: {boundaries}*
`Sentence Masking`	Masked sentences to be reconstructed	*Fill the masks, writing new sentences to be coherent with the context. Format your output according to this JSON schema: {{"MASK-0": <sentence>}, ... }. Article with gaps: {masked_text}.*
`Word Masking`	Masked words spans to be reconstructed	*Fill the masks, writing new word spans to be coherent with the context. Format your output according to this JSON schema: {{"MASK-0": <word-span>}, ... }. Article with gaps: {masked_text}.*
`Sentence Rewriting`	A sentence to rewrite	*Rewrite this sentence in your own words. Sentence: {sentence}.*
`Combined`	Combines any of the previous extractors	*Write a text similar to this one: {document}, whose summary is {summary}, using the following nouns: {nouns}; and entities: {entities}.*

Table 4. The extractors that TextMachina currently offers.

to paraphrase human texts, e.g., OpenLLMText [6] and OpenGPTText [7]. Datasets like AuTexTification [23] and SeqXGPT [30] leverage human text prefixes at word or sentence level.

TextMachina covers these and more scenarios by providing an `Extractor` interface that can be used to add new extractors by implementing the `extract` and `prepare_human` methods. Currently, we offer eleven types of extractors, depicted in Table 4, that can be linked in the prompt templates to automatically fill them with information from datasets. These extractors can be parameterized too, e.g., to select the number of sentences to be extracted in `SentencePrefix` or the amount of words to be written between the span boundaries in `WordGap`.

TextMachina automatically handles the prompt inputs, cleaning them to avoid breaking the prompt format, and truncating to avoid surpassing the maximum length allowed by LLMs. The extractors designed to generate continuations, `SentencePrefix` and `WordPrefix`, also remove the extracted prefix from the original human sources to avoid context biases, thus ensuring that both the human text and the MGT are continuations of the same prefix. Note that these extractors are also useful to prompt non-instructed LLMs, by just using the templates *{sentences}* or *{words}* without specifying additional instructions.

4.4. Dataset Generation

Different types of MGT datasets involve different generation methodologies. For instance, building a dataset aimed for boundary detection such as Subtask C in SemEval-2024 Task 8 [31] requires prepending human texts to the generations, while compiling datasets for MGT detection like AuTexTification [23] need human and generated texts to be handled independently. To abstract this, TextMachina provides a `DatasetGenerator` interface to implement different generators of MGT datasets. It accepts any human dataset in the HuggingFace Dataset format [14], and generates a labeled dataset for a given task using the same format, including metadata such as a user-defined domain, prompt, and configuration. Currently, TextMachina offers generators to build datasets for the four most popular MGT-related tasks: MGT detection, model attribution, boundary detection, and mixcase detection.

There are various strategies for generating datasets using these generators. While they are intuitive for detection and attribution, they might not be as intuitive for boundary and mixcase tasks. To build datasets for boundary tasks, users can use `SentencePrefix` and `WordPrefix` extractors to fill the prompt templates with the first k random sentences or words from human texts, and the boundary generator will concatenate the human prefix with the LLM's completion. Here, it is crucial to employ a random k value. Otherwise, the generated datasets might exhibit length biases, causing boundary detection models to merely count sentences or words. TextMachina currently allows to use three strategies to build mixcase datasets. One of them is to use `SentenceGap` as extractor, thus randomly extracting several sentence boundaries for an LLM to write sentences in between. This is the strategy depicted in Figure 3. The other two strategies are rewriting and masking. For rewriting, users can use the `SentenceRewriting` extractor, which randomly samples several sentences of the texts to be rewritten by an LLM. For masking, the `SentenceMasking` and `WordMasking` extractors can be employed. These extractors randomly replace sentences or words spans by a mask token, and an LLM has to fill all the masks at once. Instead of relying only on a boundary of one preceding and one succeding sentence/word span, as in `SentenceGap` and `WordGap`, this masking strategy allows the LLMs to use all the text as context, thus generating more coherent text. However, this task requires to generate a JSON output and generate all the masks, which is only addressable by highly-capable LLMs such as *GPT-4*.

4.5. Constrainers

In TextMachina, we define a constrainer as any kind of logic that infers something from a human dataset and constrains the LLM's decoding parameters accordingly. For instance, length constrainers automatically infer the token length of the texts and return maximum and minimum number of tokens accordingly. One could even analyze the level of formality and diversity in human texts and infer the LLM's temperature parameter to generate texts similar to those in the dataset. In TextMachina, users can define any kind of constrainer to automatically estimate decoding parameters. Currently, only length constrainers are provided by TextMachina as an attempt to alleviate length biases, avoiding large differences in token-length between generated and human texts.

4.6. Post-processing

Following the best practices in the field, TextMachina applies a set of post-processing functions by default, both to the generated and the human texts, aimed at improving the quality of any MGT dataset and preventing common biases and artifacts. These post-processing steps are carried out in the following order to ensure they are applied correctly in a cohesive manner.

Language filter. TextMachina mitigates language bias using `fastText` [12] language identification models to apply language filtering on the generated texts, discarding all the texts in a language different from the human texts.

Fix encoding. TextMachina addresses encoding bias by fixing mojibake and Unicode errors with `ftfy` [26].

Remove disclosure tokens and patterns. TextMachina ensures that no special tokens such as `[BOS]`, `[PAD]`, or `[EOS]` appear in the generated text and removes predefined disclosure patterns like "*As a language model, ...*" to minimize disclosure biases.

Remove leading and trailing whitespaces. LLMs are prone to generating texts wrapped with whitespaces. TextMachina removes any sequence of leading and trailing whitespaces both in the generations and the human texts.

Truncate. We mitigate length biases via a truncation algorithm to ensure that every class has a similar token-length distribution. This algorithm first groups texts of similar token-lengths in each class and then truncates them, thus removing as little text as possible. Moreover, all the texts containing less than 10 tokens are discarded, since typically it is not enough to correctly discriminate between generated and human. Truncation is not applied for boundary detection.

Remove empty texts. Under specific circumstances, LLMs can generate empty texts. TextMachina removes all the empty texts both from the generated and human texts.

Metric	Tasks	Description
MAUVE [21]	detection attribution	Measure divergences between all the classes using quantized embeddings of a small pre-trained language model.
Text Perplexity	detection attribution boundary	Average per-class perplexity. For boundary it treats human and generated segments separately.
Repetition & Diversity [27]	detection attribution boundary	$\text{rep}_n(y) = 100 \times (1.0 - \frac{\lvert\text{unique } n\text{-grams}(y)\rvert}{\lvert\text{total } n\text{-grams}(y)\rvert}), n \in \{2,3,4\}$ $\text{diversity}(y) = \prod_{n=2}^{4}(1.0 - \frac{\text{rep}_n(y)}{100})$ For boundary it treats human and generated segments separately.
Classification model	detection attribution boundary	For detection and attribution it is a logistic regression with bag-of-word and bag-of-character features. For boundary, it predicts the point with maximal difference in readability scores between a prefix and suffix.
Token classification model	boundary mixcase	A HuggingFace model for token classification. The tokens of human fragments are labeled as *human* and the generated ones as *generated*. Precision, recall, and F_1 between predictions and references are computed using the *seqeval* framework [18]

Table 5. Detailed description of the metrics offered by TextMachina.

Remove duplicates. It is possible that some generations are identical to human texts in the dataset, especially when dealing with shorter texts such as tweets or chat responses. For boundary detection tasks, TextMachina removes all the duplicated texts. For detection and attribution tasks, all the duplicated texts within and across labels are removed. This way, we eliminate any label noise by making the text-label pairs disjoint.

Remove generation errors. Even though TextMachina attempts to recover from server side errors, sometimes it is not possible to generate a completion even after several retries, where TextMachina discards it.

4.7. Metrics

TextMachina provides a set of metrics to assess task difficulty and dataset quality as a way to rapidly identify failure modes. For instance, if the diversity of the generated texts is too low, one may wish to modify the temperature parameter or discard the LLM. The current supported metrics are described in Table 5. These metrics can be applied both in the interactive phase and after generating a whole dataset. Five metrics are currently provided, depending on the task type: (i) MAUVE [21] to measure distributional distances between classes, (ii) Repetition and Diversity [27] to quantify text degeneration by computing ratios between unique and total n-grams, (iii) the performance of a text classification model for document-level tasks, (iv) the performance of a token classification model for boundary and mixcase tasks, and (v) per-class averaged text perplexity.

5. TextMachina in the Community

TextMachina has been used to build high-quality datasets comprised of MGT in multiple languages, styles, and domains, from different LLMs and decoding strategies.

Initial versions of TextMachina built the MGT detection and attribution datasets of the AuTexTification shared task, which are publicly available in HuggingFace and have been downloaded more than two thousand times[2]. The AuTexTification datasets contain more than 160,000 texts in English and Spanish, from BLOOM and GPT models with different parameter scales, and five domains. The quality of these datasets has been assessed by more than one hundred international teams that participated in the workshop, which fostered the development of robust models for MGT detection and attribution [22].

[2] https://huggingface.co/datasets/symanto/autextification2023

TextMachina has also been leveraged to build the MGT detection and attribution datasets of the IberAuTexTification shared task, to be presented at the Iberian Languages Evaluation Forum (IberLEF 2024)[3]. These datasets encompass over 100,000 texts generated by state-of-the-art LLMs in languages from the Iberian Peninsula. Furthermore, TextMachina was employed to create datasets for exploring the generalization capabilities of supervised MGT detectors across LLM's families and parameter scales [24].

Beyond the mentioned use cases, our framework can be used, with proper configurations, to build any existing MGT dataset in the literature. Overall, there is a large interest in building MGT datasets using TextMachina, which is reflected by more than three thousand downloads of the package in PyPI the first two months.

6. Conclusion and Future Work

We presented TextMachina, a modular Python framework that provides a comprehensive pipeline composed of a set of tools aimed to construct high-quality, unbiased datasets for MGT-related tasks such as detection, attribution, and boundary detection. Among these tools, TextMachina provides (i) dataset generators to build several kinds of MGT datasets, (ii) an interface to integrate any LLM, (iii) a set of extractors to fill prompt templates with information from human text datasets, (iv) constrainers to automatically infer LLM decoding hyperparameters, (v) post-processing functions, and (vi) a user-friendly CLI to generate and explore datasets. We have also shown common biases that could be artificially introduced in MGT datasets and how we can help the users to prevent them. TextMachina has proved its potential as a reliable tool to create MGT datasets used in shared tasks with more than one hundred participating teams. Future work includes adding new capabilities to TextMachina, e.g., supporting additional LLM providers and addressing any new challenge in the field.

References

[1] Antoun, W., Sagot, B., Seddah, D., 2023. From text to source: Results in detecting large language model-generated content. arXiv preprint arXiv:2309.13322 .

[2] Bakhtin, A., Gross, S., Ott, M., Deng, Y., Ranzato, M., Szlam, A., 2019. Real or fake? learning to discriminate machine from human generated text. arXiv preprint arXiv:1906.03351 .

[3] Brooker, M., 2015. Exponential backoff and jitter. URL: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/.

[4] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. Advances in neural information processing systems 33, 1877–1901.

[5] Chalkidis, I., Fergadiotis, E., Malakasiotis, P., Androutsopoulos, I., 2019. Large-scale multi-label text classification on EU legislation, in: Korhonen, A., Traum, D., Màrquez, L. (Eds.), Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Florence, Italy. pp. 6314–6322. URL: https://aclanthology.org/P19-1636, doi:10.18653/v1/P19-1636.

[6] Chen, Y., Kang, H., Zhai, V., Li, L., Singh, R., Raj, B., 2023a. Token prediction as implicit classification to identify LLM-generated text, in: Bouamor, H., Pino, J., Bali, K. (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore. pp. 13112–13120. URL: https://aclanthology.org/2023.emnlp-main.810, doi:10.18653/v1/2023.emnlp-main.810.

[7] Chen, Y., Kang, H., Zhai, V., Li, L., Singh, R., Ramakrishnan, B., 2023b. Gpt-sentinel: Distinguishing human and chatgpt generated content. arXiv preprint arXiv:2305.07969 .

[8] Eloundou, T., Manning, S., Mishkin, P., Rock, D., 2023. Gpts are gpts: An early look at the labor market impact potential of large language models. arXiv preprint arXiv:2303.10130 .

[9] He, X., Shen, X., Chen, Z., Backes, M., Zhang, Y., 2023. Mgtbench: Benchmarking machine-generated text detection. arXiv preprint arXiv:2303.14822 .

[10] Henderson, P., Li, X., Jurafsky, D., Hashimoto, T., Lemley, M.A., Liang, P., 2023. Foundation models and fair use. arXiv preprint arXiv:2303.15715 .

[11] Ippolito, D., Duckworth, D., Callison-Burch, C., Eck, D., 2020. Automatic detection of generated text is easiest when humans are fooled, in: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J. (Eds.), Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, Online. pp. 1808–1822. URL: https://aclanthology.org/2020.acl-main.164, doi:10.18653/v1/2020.acl-main.164.

[3] https://sites.google.com/view/iberautextification/home

[12] Joulin, A., Grave, E., Bojanowski, P., Mikolov, T., 2017. Bag of tricks for efficient text classification, in: Lapata, M., Blunsom, P., Koller, A. (Eds.), Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, Association for Computational Linguistics, Valencia, Spain. pp. 427–431. URL: https://aclanthology.org/E17-2068.

[13] Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., et al., 2023. Chatgpt for good? on opportunities and challenges of large language models for education. Learning and Individual Differences , 102274.

[14] Lhoest, Q., Villanova del Moral, A., Jernite, Y., Thakur, A., von Platen, P., Patil, S., Chaumond, J., Drame, M., Plu, J., Tunstall, L., Davison, J., Šaško, M., Chhablani, G., Malik, B., Brandeis, S., Le Scao, T., Sanh, V., Xu, C., Patry, N., McMillan-Major, A., Schmid, P., Gugger, S., Delangue, C., Matussière, T., Debut, L., Bekman, S., Cistac, P., Goehringer, T., Mustar, V., Lagunas, F., Rush, A., Wolf, T., 2021. Datasets: A community library for natural language processing, in: Adel, H., Shi, S. (Eds.), Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic. pp. 175–184. URL: https://aclanthology.org/2021.emnlp-demo.21, doi:10.18653/v1/2021.emnlp-demo.21.

[15] Liu, Y., Han, T., Ma, S., Zhang, J., Yang, Y., Tian, J., He, H., Li, A., He, M., Liu, Z., et al., 2023. Summary of chatgpt/gpt-4 research and perspective towards the future of large language models. arXiv preprint arXiv:2304.01852 .

[16] Macko, D., Moro, R., Uchendu, A., Lucas, J., Yamashita, M., Pikuliak, M., Srba, I., Le, T., Lee, D., Simko, J., Bielikova, M., 2023. MULTITuDE: Large-scale multilingual machine-generated text detection benchmark, in: Bouamor, H., Pino, J., Bali, K. (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore. pp. 9960–9987. URL: https://aclanthology.org/2023.emnlp-main.616.

[17] Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D., Finn, C., 2023. Detectgpt: Zero-shot machine-generated text detection using probability curvature. International Conference on Machine Learning .

[18] Nakayama, H., 2018. seqeval: A python framework for sequence labeling evaluation. URL: https://github.com/chakki-works/seqeval. software available from https://github.com/chakki-works/seqeval.

[19] Nasr, M., Carlini, N., Hayase, J., Jagielski, M., Cooper, A.F., Ippolito, D., Choquette-Choo, C.A., Wallace, E., Tramèr, F., Lee, K., 2023. Scalable extraction of training data from (production) language models. arXiv preprint arXiv:2311.17035 .

[20] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Gray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., Lowe, R., 2022. Training language models to follow instructions with human feedback, in: Oh, A.H., Agarwal, A., Belgrave, D., Cho, K. (Eds.), Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=TG8KACxEON.

[21] Pillutla, K., Swayamdipta, S., Zellers, R., Thickstun, J., Welleck, S., Choi, Y., Harchaoui, Z., 2021. MAUVE: Measuring the gap between neural text and human text using divergence frontiers, in: Beygelzimer, A., Dauphin, Y., Liang, P., Vaughan, J.W. (Eds.), Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=Tqx7nJp7PR.

[22] Przybyła, P., Duran-Silva, N., Egea-Gómez, S., 2023. I've seen things you machines wouldn't believe: Measuring content predictability to identify automatically-generated text, in: Proceedings of the Iberian Languages Evaluation Forum (IberLEF 2023). CEUR Workshop Proceedings, CEUR-WS, Jaén, Spain.

[23] Sarvazyan, A.M., González, J.Á., Franco-Salvador, M., Rangel, F., Chulvi, B., Rosso, P., 2023a. Overview of autextification at iberlef 2023: Detection and attribution of machine-generated text in multiple domains. Sociedad Española de Procesamiento del Languaje Natural (SEPLN) 71, 275–288.

[24] Sarvazyan, A.M., González, J.A., Franco-Salvador, M., Rosso, P., 2023b. Supervised machine-generated text detectors: Family and scale matters, in: Information Access Evaluation meets Multilinguality, Multimodality, and Visualization, Springer International Publishing.

[25] Shamardina, T., Mikhailov, V., Chernianskii, D., Fenogenova, A., Saidov, M., Valeeva, A., Shavrina, T., Smurov, I., Tutubalina, E., Artemova, E., 2022. Findings of the the ruatd shared task 2022 on artificial text detection in russian. arXiv preprint arXiv:2206.01583 .

[26] Speer, R., 2019. ftfy. Zenodo. URL: https://doi.org/10.5281/zenodo.2591652, doi:10.5281/zenodo.2591652. version 5.5.

[27] Su, Y., Lan, T., Wang, Y., Yogatama, D., Kong, L., Collier, N., 2022. A contrastive framework for neural text generation, in: Oh, A.H., Agarwal, A., Belgrave, D., Cho, K. (Eds.), Advances in Neural Information Processing Systems. URL: https://openreview.net/forum?id=V88BafmH9Pj.

[28] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al., 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 .

[29] Uchendu, A., Le, T., Shu, K., Lee, D., 2020. Authorship attribution for neural text generation, in: Webber, B., Cohn, T., He, Y., Liu, Y. (Eds.), Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online. pp. 8384–8395. URL: https://aclanthology.org/2020.emnlp-main.673, doi:10.18653/v1/2020.emnlp-main.673.

[30] Wang, P., Li, L., Ren, K., Jiang, B., Zhang, D., Qiu, X., 2023a. SeqXGPT: Sentence-level AI-generated text detection, in: Bouamor, H., Pino, J., Bali, K. (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore. pp. 1144–1156. URL: https://aclanthology.org/2023.emnlp-main.73.

[31] Wang, Y., Aji, A.F., Shelmanov, A., Whitehouse, C., Ivanov, P., Mansurov, J., Su, J., Mahmoud, T., Afzal, O.M., Nakov, P., 2024. Semeval-2024 task 8: Multigenerator, multidomain, and multilingual black-box machine-generated text detection. https://github.com/mbzuai-nlp/SemEval2024-task8.

[32] Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., Whitehouse, C., Afzal, O.M., Mahmoud, T., Aji, A.F., et al., 2023b. M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection. arXiv preprint arXiv:2305.14902 .

[33] Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., Choi, Y., 2019. Defending against neural fake news. Advances in neural information processing systems 32.

[34] Zhang, Q., Gao, C., Chen, D., Huang, Y., Huang, Y., Sun, Z., Zhang, S., Li, W., Fu, Z., Wan, Y., Sun, L., 2024. Llm-as-a-coauthor: Can mixed human-written and machine-generated text be detected? arXiv:2401.05952.